INSTITUTO FEDERAL DE MINAS GERAIS

BACHARELADO EM SISTEMAS DE INFORMAÇÃO

CARLOS HENRIQUE LIMA

EXERCÍCIO 10 – LP1

SÃO JOÃO EVANGELISTA

2025

1 – Desenvolva uma aplicação em Java que implemente uma classe responsável pela criação e fornecimento de conexões com um banco de dados MySQL, utilizando o driver JDBC do MySQL conforme visto em sala de aula. Essa classe deverá possuir atributos estáticos e privados para armazenar os parâmetros de conexão com o banco de dados, incluindo o driver JDBC, a URL de conexão, o usuário e a senha, garantindo o encapsulamento adequado dessas informações.

A classe deverá disponibilizar um método estático e público responsável por instanciar e retornar um objeto do tipo Connection, realizando o carregamento do driver, o estabelecimento da conexão com o banco de dados e o tratamento das exceções que possam ocorrer durante esse processo. O código desenvolvido deve seguir boas práticas de programação em Java e demonstrar claramente o uso do método estático para obtenção da conexão a partir de outra classe.

```
package exercicio10;
import java.sql.*;

/**
 *
 * @author carlo
 */
public class Conexoes {

    private      static      final      String      url      =
"jdbc:mysql://localhost/exercicio10";
    private static final String user = "root";
    private static final String pwd = "kaka";

    public static Connection getConnection(){
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            return   DriverManager.getConnection(url,   user,
pwd);
        } catch (ClassNotFoundException e) {
            throw new RuntimeException("Erro: Driver JDBC não
encontrado. " + e.getMessage());
        } catch (SQLException e) {
            throw new RuntimeException("Erro na conexao com
banco de dados: " + e.getMessage());
        }
    }
}
```

```
create database exercicio10;
use exercicio10;

create table clientes (
id_cliente int primary key auto_increment,
nome varchar(50)
);

insert into clientes (nome) values
('Carlos Lima'),
('Allison Pereira'),
('Pedro Rodrigues'),
('Fernando Marcos'),
('Gustavo Antonio');
```

2 – Desenvolva uma classe denominada Cliente, responsável por representar e manipular os dados de um cliente no sistema. A classe deverá conter um construtor, métodos setters e getters para seus atributos, além de métodos que implementem as operações básicas de persistência de dados no banco de dados, conforme os conceitos trabalhados em sala de aula.

A classe Cliente deverá possuir os seguintes métodos: inserir(), que não recebe parâmetros e utiliza os dados do próprio objeto para realizar a inserção no banco de dados; alterar(Cliente novoCliente), que recebe como parâmetro um objeto contendo os novos dados a serem atualizados; carregar(int id), responsável por receber o identificador do cliente e carregar suas informações a partir do banco de dados; e excluir(), que não recebe parâmetros e remove do banco de dados o cliente representado pelo objeto atual. O código deve seguir boas práticas de programação orientada a objetos e utilizar a classe de conexão desenvolvida anteriormente.

```
package exercicio10;

import java.sql.*;

/**
 *
 * @author carlo
 */
public class Cliente {

    private int id;
    private String nome;
```

```
    public Cliente() {
    }

    public Cliente(int id, String nome) {
        this.id = id;
        this.nome = nome;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }

    public void inserir() {
        String sql = "insert into clientes (nome) values (?)";
        try (Connection conn = Conexoes.getConnection();
PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setString(1, this.nome);
            stmt.executeUpdate();
            System.out.println("Cliente inserido com
sucesso!");
        } catch (SQLException e) {
            throw new RuntimeException("Erro ao inserir: " +
e.getMessage());
        }
    }

    public void alterar(Cliente novoCliente) {
        String sql = "update clientes set nome = ? where
id_cliente = ?";
        try (Connection conn = Conexoes.getConnection();
PreparedStatement stmt = conn.prepareStatement(sql)) {

            stmt.setString(1, novoCliente.getNome());
            stmt.setInt(2, this.id);
            stmt.executeUpdate();

            this.nome = novoCliente.getNome();
            System.out.println("Cliente do ID " + this.id + "
alterado");
```

```
        } catch (SQLException e) {
            throw new RuntimeException("Erro ao alterar
cliente: " + e.getMessage());
        }
    }

    public void carregar(int id) {
        String sql = "select * from clientes where id_cliente
= ?";
        try (Connection conn = Conexoes.getConnection();
PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setInt(1, id);
            try (ResultSet rs = stmt.executeQuery()) {
                if (rs.next()) {
                    this.id = rs.getInt("id_cliente");
                    this.nome = rs.getString("nome");
                } else {
                    System.out.println("Cliente não
encontrado.");
                }
            }
        } catch (SQLException e) {
            throw new RuntimeException("Erro ao carregar: " +
e.getMessage());
        }
    }

    public void excluir() {
        String sql = "DELETE FROM clientes WHERE id_cliente =
?";
        try (Connection conn = Conexoes.getConnection();
PreparedStatement stmt = conn.prepareStatement(sql)) {

            stmt.setInt(1, this.id);
            stmt.executeUpdate();
            System.out.println("Cliente excluído.");

        } catch (SQLException e) {
            throw new RuntimeException("Erro ao excluir: " +
e.getMessage());
        }
    }
}
```

```
package exercicio10;

import java.util.Scanner;

/**
```

```
 *
 * @author carlo
 */
public class Exercicio10 {

    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {
        // TODO code application logic here
        Scanner scan = new Scanner(System.in);
        Cliente c = new Cliente();
        c.setNome("Carlos Henrique");
        c.inserir();

        Cliente c2 = new Cliente();
        c2.carregar(5);
        System.out.println("Nome atual: " + c2.getNome());

        Cliente c3 = new Cliente();
        c3.setNome("Carlos Lima");
        c2.alterar(c3);

        c2.excluir();
    }

}
```

Result Grid | Filter Rows:

id_cliente	nome
1	Carlos Henrique
2	Carlos Lima
3	Allison Pereira
4	Pedro Rodrigues
5	Fernando Marcos
6	Gustavo Antonio
NULL	NULL

Result Grid | Filter Rows:

id_cliente	nome
1	Carlos Henrique
2	Carlos Lima
3	Allison Pereira
4	Pedro Rodrigues
6	Gustavo Antonio
7	Carlos Henrique
NULL	NULL

```
run:
Cliente inserido com sucesso!
Nome atual: Fernando Marcos
Cliente do ID 5 alterado
Cliente excluído.
CONSTRUÍDO COM SUCESSO (tempo total: 0 segundos)
```

```java
package exercicio10;

import java.util.Scanner;

/**
 *
 * @author carlo
 */
public class Exercicio10 {

    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {
        // TODO code application logic here
        Scanner scan = new Scanner(System.in);
        Cliente c = new Cliente();
        c.setNome("Carlos Henrique");
        c.inserir();

        Cliente c2 = new Cliente();
        c2.carregar(6);
        System.out.println("Nome atual: " + c2.getNome());

        Cliente c3 = new Cliente();
        c3.setNome("Carlos Lima");
        c2.alterar(c3);

        c2.excluir();
    }

}
```

Result Grid | Filter Rows:

id_cliente	nome
1	Carlos Henrique
2	Carlos Lima
3	Allison Pereira
4	Pedro Rodrigues
6	Gustavo Antonio
7	Carlos Henrique
NULL	NULL

Result Grid | Filter Rows:

id_cliente	nome
1	Carlos Henrique
2	Carlos Lima
3	Allison Pereira
4	Pedro Rodrigues
7	Carlos Henrique
8	Carlos Henrique
NULL	NULL

```
run:
Cliente inserido com sucesso!
Nome atual: Gustavo Antonio
Cliente do ID 6 alterado
Cliente excluído.
CONSTRUÍDO COM SUCESSO (tempo total: 0 segundos)
```